

Appendix C — Cheat Sheets

The quick-reference tables you reach for once the concepts are in hand — storage layout, message types, commands, states, and the constants that define the network.

SUBJECT hathor-core · Appendix C

CHAPTER C · Appendices

BRANCH study/hathor-core-studies

EDITION 2026 · v1

RocksDB column families · P2P messages · CLI commands · Sysctl paths · Feature states ·
Handshake states · Network constants

Table of Contents

Appendix C — Cheat Sheets	3
C.1 RocksDB column families	3
C.2 P2P protocol messages	4
C.3 CLI command groups	5
C.4 Sysctl parameter paths	6
C.5 Feature-activation state machine	7
C.6 P2P handshake states	8
C.7 Network constants	8
Recap	9

Appendix C — Cheat Sheets

The chapters explain why; these tables are the what, collected for fast lookup. Nothing here is new — every table condenses material from a chapter named in its heading, verified against the source as of the branch on the cover. Use this appendix the way you would a wall chart: when you are reading the code and need the exact column-family name, the precise message string, or the value of a network constant, find it here without re-reading the chapter.

C.1 RocksDB column families

The on-disk store is split into named **column families**¹ — independent key spaces inside one database. Vertex bodies, mutable metadata, and immutable static metadata are kept apart so each can be read, written, and (in principle) tuned independently. All vertex-keyed families use the 32-byte vertex hash as the key. (Chapter 27; `transaction/storage/rocksdb_storage.py:46 – 50`.)

Column family	Constant	Key	Value	Holds
tx	<code>_CF_NAME_TX</code>	vertex hash (32 B)	binary <code>bytes(vertex)</code>	the serialized vertex body (Ch 26)
meta	<code>_CF_NAME_META</code>	vertex hash (32 B)	binary metadata	mutable <code>TransactionMetadata</code> — <code>voided_by</code> , <code>score</code> , ... (Ch 25, 32)
static-meta	<code>_CF_NAME_STATIC_META</code>	vertex hash (32 B)	JSON	immutable <code>VertexStaticMetadata</code> — <code>height</code> , <code>min-height</code> (Ch 25)
attr	<code>_CF_NAME_ATTR</code>	attribute name	flag/value	node flags incl. the crash markers <code>manager_running</code> / <code>full_node_crashed</code> (Ch 29)
migrations	<code>_CF_NAME_MIGRATIONS</code>	migration name	state	which schema migrations have run

Note. There is no in-RAM storage backend; ephemeral runs use a throwaway temporary RocksDB (`create_temp()` , `--temp-data`), not a different class. (Chapter 27.)

C.2 P2P protocol messages

Every peer-to-peer message is one line of text: `COMMAND payload` . The commands are the `ProtocolMessages` enum (Chapter 34; `p2p/messages.py:45`). Grouped by purpose:

Handshake & connection management

Message	Meaning
HELLO	open the handshake; advertise protocol/sync versions
PEER-ID	exchange and verify peer identity
READY	both sides verified; connection is live
ERROR	protocol error
THROTTLE	rate-limit notice
PING / PONG	liveness check / reply
GET-PEERS / PEERS	request / send known peer addresses

Best-chain & generic data

Message	Meaning
GET-BEST-BLOCKCHAIN / BEST-BLOCKCHAIN	query / report the peer's best chain
GET-DATA / DATA	request / send a specific vertex
NOT-FOUND	requested vertex is unknown
RELAY	enable/disable live push of new vertices

Sync-v2 — block backbone (Chapter 35)

Message	Meaning
GET-BEST-BLOCK / BEST-BLOCK	am I behind?
GET-PEER-BLOCK-HASHES / PEER-BLOCK-HASHES	n-ary common-ancestor search

Message	Meaning
GET-NEXT-BLOCKS / GET-PREV-BLOCKS	request a run of blocks
BLOCKS / BLOCKS-END	streamed blocks / stream end
STOP-BLOCK-STREAMING	abort a block stream

Sync-v2 — transactions & mempool (Chapter 35)

Message	Meaning
GET-TRANSACTIONS-BFS	request the txs a block confirms (BFS)
TRANSACTION / TRANSACTIONS-END	streamed tx / stream end
STOP-TRANSACTIONS-STREAMING	abort a tx stream
GET-TIPS / TIPS / TIPS-END	mempool tip discovery
GET-MEMPOOL / MEMPOOL-END	mempool sync

Nano-contract state sync (Chapter 39)

Message	Meaning
GET-BLOCK-NC-ROOT-ID / BLOCK-NC-ROOT-ID	query a block's nano-contract state root
GET-NC-DB-NODE / NC-DB-NODE	fetch a node of the contract-state trie

Rot warning. The enum still contains sync-v1-era messages (GET-NEXT , NEXT) — sync-v1 has been removed and these are not used. (Chapter 34/35.)

C.3 CLI command groups

`hathor-cli <command>` dispatches through a name→module dictionary (Chapter 21; `hathor_cli/main.py`). The ~30 commands by group:

Group	Commands	Purpose
<code>hathor</code>	<code>run_node</code> , <code>gen_peer_id</code> , <code>top</code>	run a node; generate a peer id; CPU-profiler viewer

Group	Commands	Purpose
mining	<code>run_miner</code> , <code>run_stratum_miner</code> , <code>run_merged_mining</code>	external miners (Ch 37)
wallet	<code>gen_kp_wallet</code> , <code>gen_hd_words</code>	create a keypair wallet; generate HD seed words (Ch 40)
multisig	<code>gen_multisig_address</code> , <code>spend_multisig_output</code> , <code>tx_signature</code>	multisig tooling
oracle	<code>oracle-create-key</code> , <code>oracle-get-pubkey</code> , <code>oracle-encode-data</code>	oracle data signing
side-dag	<code>run_node_with_side_dag</code> , <code>gen_poa_keys</code> , <code>gen_genesis</code>	side-DAG / proof-of- authority (Ch 32)
events	<code>reset-event-queue</code>	clear the event queue (Ch 30)
features	<code>reset-feature-settings</code>	clear feature-activation state (Ch 38)
docs	<code>generate_openapi_json</code>	API docs
tests	<code>gen_rand_tx</code> , <code>gen_twin_tx</code>	test-data generators
dev	<code>shell</code> , <code>quick_test</code> , <code>db_export/import</code> , <code>nc_dump</code> , <code>check_blueprint</code> , ... (11 total)	developer/debug tools

C.4 Sysctl parameter paths

Runtime control exposes a tree of dotted-path parameters over a socket (Chapter 41; `hathor/sysctl/`). Address a parameter as `path` (get) or `path=value` (set, JSON value). Representative paths:

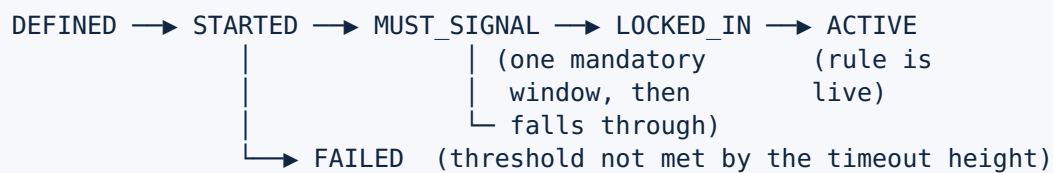
Path	Get / Set	Effect
<code>core</code>	—	the <code>HathorManagerSysctl</code> root for node-wide knobs (debugging, profiling)
<code>core.features</code>	get	feature-activation state
<code>p2p.max_enabled_sync</code>	get / set	cap on simultaneously-syncing peers

Path	Get / Set	Effect
p2p.available_sync_versions	get	sync versions on offer (v2 in practice)
p2p.kill_connection	set	drop a specific peer connection
storage	—	the StorageSysctl subtree
ws	—	websocket controls (if enabled)

Three entry points feed the same command tree: the socket (--sysctl), a batch init file (--sysctl-init-file), and the SIGUSR2 signal (reads a per-pid FIFO; only signal-safe setters allowed). No authentication — access is governed by socket permissions. (Chapter 41.)

C.5 Feature-activation state machine

A feature moves through six states as evaluation windows pass, gated on the fraction of blocks signalling its bit (Chapter 38; `feature_activation/model/feature_state.py`).



State	Meaning
DEFINED	known but not yet started (before its start height)
STARTED	the signalling window is open; blocks may vote for the bit
MUST_SIGNAL	a single window where signalling is mandatory before lock-in
LOCKED_IN	enough support gathered; activation is now scheduled
ACTIVE	the feature's rule is in force
FAILED	the timeout height passed without enough support

Key tuning constants (`feature_activation/settings.py`): `evaluation_interval = 20_160` blocks per window, `default_threshold = 18_144` (90%), `max_signal_bits = 8`.

C.6 P2P handshake states

Each connection is driven by a three-state machine in `HathorProtocol` (Chapter 34; `p2p/states/`). The transition to `READY` is symmetric — it fires only when both peers are satisfied.



State	Class	Handles
HELLO	<code>HelloState</code>	version negotiation, network match, genesis check
PEER_ID	<code>PeerIdState</code>	identity exchange & verification
READY	<code>ReadyState</code>	live connection; attaches the sync agent

C.7 Network constants

The values that define the network — every node must agree on them or it is on a different network (Chapter 22; the genesis/timing constants live in `hathorlib/conf/settings.py`). Mainnet values:

Constant	Value	What it controls
<code>AVG_TIME_BETWEEN_BLOCKS</code>	30 s	target block interval; the DAA tunes weight toward it (Ch 9)
<code>MIN_BLOCK_WEIGHT</code>	21	floor on block proof-of-work weight
<code>MIN_TX_WEIGHT</code>	14	floor on transaction (anti-spam) weight
<code>MIN_TX_WEIGHT_COEFFICIENT</code>	1.6	size term in the min-tx-weight formula
<code>MIN_TX_WEIGHT_K</code>	100	amount term in the min-tx-weight formula
<code>DECIMAL_PLACES</code>	2	smallest unit = 1/100 of a token
<code>BLOCKS_PER_HALVING</code>	1,051,200	≈ every 365 days; the reward-halving schedule
<code>HATHOR_TOKEN_UID</code>	<code>b'\x00'</code>	the native-token marker — a single null byte, not a 32-byte hash (Ch 7)
<code>MAX_DISTANCE_BETWEEN_BLOCKS</code>	4,500 s	150×30 ; weight-decay trigger when blocks stall (Ch 9)

These are read everywhere through `get_global_settings()` and never change for the life of the process — the single immutable source of truth for "what network am I on." (Chapter 22.)

Recap

Cheat sheet	Chapter	Source of truth
RocksDB column families	27	<code>transaction/storage/rocksdb_storage.py:46</code>
P2P messages	34, 35, 39	<code>p2p/messages.py:45</code>
CLI commands	21	<code>hathor_cli/main.py</code>
Sysctl paths	41	<code>hathor/sysctl/</code>
Feature states	38	<code>feature_activation/model/feature_state.py</code>
Handshake states	34	<code>p2p/states/</code>
Network constants	22, 9	<code>hathorlib/conf/settings.py</code>

These tables are the residue of the whole book — the facts that remain once the explanations have done their work and become second nature. With the three appendices in place — the **glossary** for every term, the **dependency manifest** for every package, and these **cheat sheets** for every recurring table — the master-book is complete: a junior reader can now go from the first "what is Hathor?" in Chapter 0 to reading any corner of the running node, with a reference for anything they forget along the way.

1. A column family in RocksDB is a named, independent key-value namespace within a single database — like separate tables sharing one store. Each can be configured and accessed on its own, while still committing atomically with the others. ↩